

A formalisation of Gödel’s Second Incompleteness Theorem for the Basic Recursive Arithmetic of Church and Guard

Abstract

We report a complete, axiom-free, machine-checked formalisation in Agda of Gödel’s second incompleteness theorem for Church’s *basic recursive arithmetic* (BRA), following the proof outline given by R. Guard in his 1963 paper. Beyond the formalisation itself, the work exposes a number of implicit assumptions, notational ambiguities, and minor typographical issues in Guard’s text that are worth recording explicitly. We also record an architectural simplification we have since carried out: replacing the dedicated multi-variable simultaneous-substitution machinery by nested single-variable substitution together with a numeral-inertness lemma (the internalisation of “a numeral is closed”), which removes roughly ten thousand lines. Throughout we follow Shoenfield’s notation (*Mathematical Logic*, Chapter 6), with the single substitution $code(u)$ in place of $\ulcorner u \urcorner$.

1 The formal system

1.1 Term and function syntax (Church, via Guard)

We follow Church’s basic recursive arithmetic [1] in the formulation of Guard [2], with Shoenfield’s notation [3]. Following Shoenfield §6.6, the non-logical symbols of the theory T are a constant $\mathbf{O}$, a unary symbol $\mathbf{s}$, and finitely many additional function symbols (Church’s combinators $\mathbf{o}, \mathbf{u}, \mathbf{v}, \mathbf{C}, \mathbf{R}$), together with the binary identity $=$. Variables are $\mathbf{x}_0, \mathbf{x}_1, \dots$ in alphabetical order, with symbol number $SN(\mathbf{x}_i) = 2i$.

The theory T has two further function-symbol grammars:

$$\begin{aligned} \mathbf{f} &::= \mathbf{s} \mid \mathbf{o} \mid \mathbf{u} \mid \mathbf{C}(\mathbf{g}, \mathbf{f}_1, \mathbf{f}_2) \quad (\text{unary, } \mathbf{f}, \mathbf{f}_1, \mathbf{f}_2 \in Fun_1), \\ \mathbf{g} &::= \mathbf{v} \mid \mathbf{R}(\mathbf{f}, \mathbf{g}_1, \mathbf{g}_2) \quad (\text{binary, } \mathbf{g}, \mathbf{g}_1, \mathbf{g}_2 \in Fun_2). \end{aligned}$$

Terms $\mathbf{t}$ are built from $\mathbf{O}$, variables $\mathbf{x}_i$, and applications $\mathbf{f}(\mathbf{t})$ and $\mathbf{g}(\mathbf{t}_1, \mathbf{t}_2)$.

1.2 Formulas and derivability

Formulas are built from equations between terms by negation and implication:

$$\mathbf{A}, \mathbf{B} ::= \mathbf{t}_1 = \mathbf{t}_2 \mid \neg \mathbf{A} \mid \mathbf{A} \supset \mathbf{B}.$$

We write $\vdash \mathbf{A}$ for “ $\mathbf{A}$ is a theorem of T ” ($\mathbf{A} \in Thm_T$ in Shoenfield’s notation). Bare $\mathbf{A} = \mathbf{B}$ inside $\vdash$ means $\vdash \mathbf{A} = \mathbf{B}$.

1.3 The axioms and rules, explicitly

The Hilbert calculus is exactly Guard’s Definition 7 [2], itself following Church’s Princeton lectures [1]. The Agda data type `BRA3.Deriv.Deriv` has one constructor per Guard axiom and rule and *nothing else*; we list all fourteen axiom schemes and three rules, in Guard’s numbering, with the Agda constructor name in brackets. Here $\mathbf{f} \in Fun_1$, $\mathbf{g} \in Fun_2$, $\mathbf{t}, \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{x}, \mathbf{n}$ are terms, $\mathbf{A}, \mathbf{B}, \mathbf{C}$ are formulas, and $a \in \mathbb{N}$ is a variable index.

Equational axioms (the defining equations of the combinators).

- 0 [ax_succ_nonzero] : $\neg (s(O) = O)$
- 1 [ax_o] : $o(t) = O$
- 2 [ax_u] : $u(t) = t$
- 3 [ax_v] : $v(a, b) = b$
- 8 [ax_C] : $C(g, f_1, f_2)(t) = g(f_1(t), f_2(t))$
- 9 [ax_R_base] : $R(f, g_1, g_2)(x, O) = f(x)$
- 10 [ax_R_step] : $R(f, g_1, g_2)(x, s\ n) = g_1(g_2(x, n), R(f, g_1, g_2)(x, n))$

Equality axioms.

- 4 [ax_eqTrans] : $x = y \supset . x = z \supset y = z$
- 5 [ax_eqCong₁] : $a = b \supset f(a) = f(b)$
- 6 [ax_eqCong_L] : $a = b \supset g(a, c) = g(b, c)$
- 7 [ax_eqCong_R] : $a = b \supset g(c, a) = g(c, b)$

Propositional axioms.

- 11 [axK] : $A \supset . B \supset A$
- 12 [axS] : $(A \supset (B \supset C)) \supset . (A \supset B) \supset . A \supset C$
- 13 [axNeg] : $(\neg A \supset \neg B) \supset . B \supset A$

Rules of inference.

- I [mp] : from $A \supset B$ and A , infer B .
- III [ruleInst] : from A , infer $substF(a, t, A)$ (Shoenfield's *Sub*).
- VI [ruleIndNat] : from $substF(a, O, A)$ and $A \supset substF(a, s(x_a), A)$, infer A .

Two economies of Guard's system, which we do *not* exploit (we keep all fourteen axioms as constructors for fidelity): Dana Scott showed axiom 7 derivable from the others [2, Exercise after Def. 6], and axiom 5 is derivable from the others excluding axiom 7 [2, Exercise 18]. Rule VI is stated only for induction on x_0 in Guard; induction on an arbitrary variable, and the substitutivity-of-equality rule, are derivable [2, Exercise 19].

This list is the complete trusted base: every theorem in the development, up to and including `godelIII`, is built solely from these seventeen constructors under `--safe --without-K --exact-split`, with no postulates.

1.4 Numerals and the encoding $code(u)$

For each $n \in \mathbb{N}$ the numeral is $\mathbf{k}_n := s^n(O)$ (Shoenfield's $\mathbf{k}_n$). Guard's text uses a unary internal functor `num` $\in Fun_1$, the *numeral-to-code* map: applied to a numeral, `num` produces the *encoding* of that numeral. Its defining property is the Deriv-level closure

$$\vdash \text{num}(\mathbf{k}_n) = code(\mathbf{k}_n) \quad \text{for every meta-natural } n,$$

where $code(\mathbf{k}_n)$ is the Gödel handle of the numeral $\mathbf{k}_n$. This is the T4 lemma *numEq n* (its Agda form is $\vdash \text{num}(\text{natCode } n) = codeTerm(\text{natCode } n)$, where $\text{natCode } n = \mathbf{k}_n$ and $codeTerm = code(\cdot)$ on terms).

In particular `num` does *not* act as the identity on numerals: for instance

$$\text{num}(s\ O) = code(s\ O) = \langle \mathbf{k}_{\text{tag_ap1}}, \langle \mathbf{k}_{\text{tag_s}}, O \rangle \rangle \neq s\ O.$$

The functor **num** is precisely Guard’s underline: $\underline{x} := \mathbf{num}(x)$, the code of the value of x . On a numeral k_n this is the encoding $code(k_n)$; on the recursion side it satisfies the unconditional structural closures

$$\vdash \mathbf{num}(\mathbf{O}) = \mathbf{O}, \quad \vdash \mathbf{num}(s\ t) = \langle k_{\text{tag_ap1}}, \langle k_{\text{tag_s}}, \mathbf{num}(t) \rangle \rangle$$

(the Agda lemmas **num_at_O**, **num_at_S**), so that **num** wraps each s in the same nested layout that $code(\cdot)$ uses for $s(\cdot)$, making $\vdash \mathbf{num}(k_n) = code(k_n)$ a straightforward meta-induction on n .

The encoding $code(u)$ of a designator u is defined by induction on the formation rules, using Shoenfield’s sequence notation $\langle a_0, a_1, \dots, a_n \rangle$ (§6.6). In our concrete Agda realisation the sequence operation is Church’s binary pairing functor $\pi \in Fun_2$, written $\langle a, b \rangle$, with longer sequences nesting to the right, $\langle a, b, \dots \rangle = \langle a, \langle b, \dots \rangle \rangle$; the abstract notation $\langle \cdot \rangle$ below leaves this implementation choice tacit. The encoding schema is then:

$$\begin{aligned} code(\mathbf{O}) &= \mathbf{O}, \\ code(x_k) &= \langle k_{\text{tag_var}}, k_k \rangle, \\ code(f(t)) &= \langle k_{\text{tag_ap1}}, code(f), code(t) \rangle, \\ code(g(t_1, t_2)) &= \langle k_{\text{tag_ap2}}, code(g), code(t_1), code(t_2) \rangle, \\ code(t_1 = t_2) &= \langle k_{\text{tag_eq}}, code(t_1), code(t_2) \rangle, \\ code(\neg A) &= \langle k_{\text{tag_neg}}, code(A) \rangle, \\ code(A \supset B) &= \langle k_{\text{tag_imp}}, code(A), code(B) \rangle, \end{aligned}$$

with codes for the Fun_1 and Fun_2 symbols defined analogously. The tags are concrete meta-naturals (e.g. $\text{tag_var} = 1$, $\text{tag_ap1} = 2$, $\text{tag_eq} = 10$, $\text{tag_neg} = 11$, $\text{tag_imp} = 12$, etc.).

1.5 The verifier **thmT** and the substitution functor **sub**

Guard’s verifier **thmT** $\in Fun_1$ (his **th**, Definition 16 of [2]) is internally constructed by course-of-values recursion so as to satisfy two structural closures:

Soundness-by-construction. If $code(d)$ is the encoding of a Hilbert derivation of A , then $\vdash \mathbf{thmT}(code(d)) = code(A)$.

Validating-decoder invariant. On inputs that do not encode a well-formed derivation, **thmT** returns the canonical truth $code(\mathbf{O} = \mathbf{O})$.

Its construction by course-of-values recursion — the genuinely non-trivial step of realising such recursion inside Church’s $\{s, o, u, v, C, R\}$ grammar — is the subject of §2.

The substitution functor **sub** $\in Fun_2$ is Guard’s Exercise 24[8] of [2]:

$$\vdash \mathbf{sub}(z, code(A)) = \mathbf{sbF}(\langle k_0, \mathbf{num}(z) \rangle, code(A)),$$

where **sbF** $\in Fun_2$ is the formula-level single-variable encoded substitution. The point of separating **sub** from **sbF** is that **sub** is the function which appears in Guard’s text; **sbF** is the lower-level realisation.

1.6 The diagonal lemma

Let **F** be the seed formula

$$\mathbf{F} \equiv \neg(\mathbf{thmT}(x_1) = x_0).$$

Define

$$\begin{aligned} \mathbf{H} &\equiv Sub(code(\mathbf{F}), code(x_0), code(\mathbf{sub}(x_0, x_0))), \\ i &:= Num(code(\mathbf{H})), \\ \mathbf{G} &\equiv Sub(code(\mathbf{H}), code(x_0), code(k_i)). \end{aligned}$$

Then $\mathbf{G}$ is closed in $\mathbf{x}_0$ with one free variable $\mathbf{x}_1$, and satisfies the diagonal identity

$$\vdash \text{sub}(\mathbf{k}_i, \mathbf{k}_i) = \text{code}(\mathbf{G}),$$

which is the Agda lemma `Thm14F.diag_term_eq`.

2 Course-of-values recursion inside Church’s recursor

The verifier **thmT** is defined by recursion on the Gödel number of a derivation, where the value at a node depends on the values at *all* smaller indices (the sub-derivations) — *course-of-values* recursion. Church’s grammar provides only the primitive recursor **R**. The reduction has a routine outline with one genuinely delicate point. The *first step* — express bounded *iteration* f^n of a unary f in terms of **R**, and encode the whole history (the table of sub-values) as a single threaded value — is the standard course-of-values move, and is not surprising. What is *not* routine, and is the subject of this section, is that Church’s **R** is not the textbook recursor: it returns the recursion result in a non-standard argument slot, so even the iteration combinator f^n must be synthesised by a *nested R* whose defining equation is proved *inside T* by induction, not by computation; and the read-off must be shown independent of surplus recursion fuel. Those three points — the slot mismatch, the internally-proved iteration combinator, and fuel-stability — are the genuine content.

2.1 Church’s recursor is not the textbook one

The textbook primitive recursor satisfies $\rho(\mathbf{x}, \mathbf{O}) = g(\mathbf{x})$ and $\rho(\mathbf{x}, \mathbf{s}\ \mathbf{n}) = h(\mathbf{n}, \rho(\mathbf{x}, \mathbf{n}))$, feeding the recursion result to h in a fixed argument slot. Church’s **R** (axioms 9, 10 of §1.3) instead satisfies

$$\mathbf{R}(\mathbf{f}, \mathbf{g}_1, \mathbf{g}_2)(\mathbf{x}, \mathbf{s}\ \mathbf{n}) = \mathbf{g}_1(\mathbf{g}_2(\mathbf{x}, \mathbf{n}), \mathbf{R}(\mathbf{f}, \mathbf{g}_1, \mathbf{g}_2)(\mathbf{x}, \mathbf{n})),$$

so the recursion result enters $\mathbf{g}_1$ as its *second* argument, while $\mathbf{g}_2(\mathbf{x}, \mathbf{n})$ supplies an auxiliary first argument. This shape is convenient for some recursions and awkward for others; in particular it does *not* directly give “iterate a unary $\mathbf{f}$,” which is what course-of-values needs.

2.2 The history-as-value encoding

Following Church’s pairing functors J, K, L (Definition 6 of [2]; the realisation uses Church’s π , written $\langle a, b \rangle$, with projections Fst, Snd), the history of the values at nodes $0, \dots, n$ is encoded as the right-nested list

$$\text{table}_n = \langle v_n, v_{n-1}, \dots, v_0 \rangle = \langle v_n, \langle v_{n-1}, \dots \langle v_0, \mathbf{O} \rangle \rangle \rangle,$$

and the recursion carries a *state*

$$\text{state}_n = \langle \mathbf{k}_n, p, \text{table}_n \rangle.$$

This threads three things: the counter $\mathbf{k}_n$ (the numeral of the current node n); an immutable parameter p , recoverable as $Fst(Snd(\text{state}))$ at every step — this is how the step function reads a fixed parameter *without* it being a BRA argument; and the full table table_n . (The parameter p is what the Agda calls the *spec*: for **sbt/sbf** it carries the substitution specification $\langle k, S \rangle$, and for **thmT** it is an unused dummy $\mathbf{O}$; we write it p .) At each step the new value $v_{n+1} = \text{stepFun}(Fst\ \text{state}, Snd\ \text{state})$ is consed onto the table; the final answer is read off the head,

$$\mathbf{thmT} = \text{Post readOff (cov_spec baseFun stepFun)}.$$

This is the Agda module `T4.CoVSpec` (the “spec” in the name is this parameter p), built on `BRA3.CourseOfValues`.

2.3 The nested-**R** trick

The crux is that the state recursion needs to *apply a unary function to the previous state* — i.e. an iteration combinator $iter\ f$ with $iter\ f(\mathbf{x}, \mathbf{k}) = f^{\mathbf{k}}(\mathbf{x})$. But Church’s **R** delivers the recursion result into $\mathbf{g}_1$ ’s *second* slot, whereas the naive lift $\mathbf{R}\ f\ \mathbf{v}\ \mathbf{v}$ can read it only via the *first*. Bridging the two requires a binary $\mathbf{g}_1$ with $\mathbf{g}_1(\mathbf{a}, \mathbf{b}) = f(\mathbf{b})$ — a function that ignores its first argument and applies f to its second. This is itself built as *another* nested **R**,

$$iter_step_fun\ f = \mathbf{R}\ (compose_1\ f\ \mathbf{o})\ (\mathbf{R}\ (compose_1\ f\ \mathbf{s})\ \mathbf{v}\ \mathbf{v})\ \mathbf{v},$$

and the defining equation $\mathbf{g}_1(\mathbf{a}, \mathbf{b}) = f(\mathbf{b})$ is then *proved inside T by induction* (**ruleIndNat**, Rule VI), not by computation. This use of the object-level induction rule to establish a meta-level recursion scheme is exactly what makes the step “non-trivial to do for Church’s system”: the iteration combinator is not primitive, it is a derived object whose correctness is an internal theorem.

2.4 Fuel and stability

Church’s **R** recurses down to a *given* numeral, so *cov_spec* must be run with an explicit *fuel* argument: a numeral bounding how far the history is built. Evaluating $\mathbf{thmT}(y)$ needs the recursion run to at least node y — the value for y sits at depth y in the history — and since no tight bound on the required depth is known in advance, one runs with *surplus* fuel and must show the answer does not depend on it.

That is the content of the *stability* lemmas (**T4.StabilityNatFuel**, **T4.Stability**): once the counter has reached the target node n , building further history leaves the value already recorded at n unchanged,

$$readOff(state_{n'}) = readOff(state_n) \quad \text{for all } n' \geq n.$$

Equivalently, the read-off at a node is insensitive to surplus fuel above it. Together with the per-projection preservation lemmas, this is what lets the finitely-many per-tag closure equations of $\mathbf{thmT}(\mathbf{thmT_at_ax}_0, \dots, \mathbf{thmT_at_mp}, \mathbf{thmT_at_sb})$ be stated and discharged for an arbitrary sufficiently-large fuel, rather than a hand-computed one.

In sum: course-of-values recursion is *representable* in BRA, but not for free. Its representation requires (i) a history-carrying state threaded through Church’s non-standard **R**, (ii) the iteration combinator of the nested-**R** trick, whose correctness is an internal induction, and (iii) the fuel-stability lemmas above. This is the foundation on which **thmT**, **sbt**, **sbF**, **num**, and **Eval** (of the Berry route) all rest.

3 The main theorem

Theorem 3.1 (Gödel II for BRA, formalised). *Let*

$$Con_T \equiv \neg(\mathbf{thmT}(\mathbf{x}_0) = code(\mathbf{O} = \mathbf{s}\ \mathbf{O})).$$

This is an open formula, with the single free variable $\mathbf{x}_0$. BRA has no object quantifiers (§1.3): a theorem is an open formula, read as implicitly universally quantified over its free variables — $\vdash \mathbf{A}(\mathbf{x}_0)$ delivers, by Rule III, every closed instance $\vdash \mathbf{A}(\mathbf{t})$. So Con_T , as that single open formula, is the internal consistency statement: for every term $\mathbf{x}_0$, $\mathbf{x}_0$ does not code a derivation of $\mathbf{O} = \mathbf{s}\ \mathbf{O}$. Then

$$godelIII : \quad \vdash Con_T \implies \vdash (\mathbf{O} = \mathbf{s}\ \mathbf{O}).$$

The arrow “ $\Rightarrow$ ” is a *meta-level* implication, not an object connective. What is proved is a *meta-statement* — equivalently, a total function transforming any closed derivation of Con_T into a closed derivation of $\mathbf{O} = \mathbf{s}\mathbf{O}$ — and *not* a single closed Hilbert-style derivation of the object implication “ $Con_T \supset (\mathbf{O} = \mathbf{s}\mathbf{O})$ ”. This is exactly what the Agda type records: `godelIII` is an arrow of the meta-language Agda, with `Deriv` the (object) derivability relation and `->` Agda’s function arrow:

```
godelIII : Deriv ConSchema -> Deriv P_false
  where ConSchema = neg (eqF (ap1 thmT (var 0)) codeFalse)
        P_false    = atomic (eqn 0 (ap1 s 0))
```

and is shipped in `T4/Thm/Thm14GodelIII.agda` (241 LoC, fresh chain to the theorem 7.2s), with no postulates, no holes, and the global options `--safe --without-K --exact-split`. The full T4 development has zero `postulate` declarations anywhere in its $\sim 50\,000$ lines.

4 Structure of the proof

The proof follows Guard’s five-step decomposition (*guard15.pdf* p. 17), with Step 5 expressed as an explicit composition of two encoded applications of modus ponens. Throughout, $\mathbf{x}$ ranges over T -terms at the meta level, and is *universally* quantified at the meta-level (so $\vdash \mathbf{A}(\mathbf{x})$ below stands for “ $\vdash \mathbf{A}$ as an open formula with free variable $\mathbf{x}$ ” — giving every closed instance via Rule III).

Step 1. By the internal Theorem 13 for unary functionals,

$$\vdash (\mathbf{thmT}(\mathbf{x}) = \mathbf{k}_j) \supset (\mathbf{thmT}(D_{\mathbf{thmT}}(\mathbf{x})) = \mathit{code}(\mathbf{thmT}(\mathbf{num}(\mathbf{x})) = \mathbf{num}(\mathbf{k}_j))),$$

where $D_{\mathbf{thmT}} \in Fun_1$ is the Hilbert-internal handle for the verifier $\mathbf{thmT}$ supplied by Theorem 12, and j is the Goedel-handle meta-natural of $\mathbf{G}$.

Step 2. By the internal Theorem 13 for the binary `sub`:

$$\vdash \mathbf{thmT}(D_{\mathbf{sub}}(\mathbf{k}_i, \mathbf{k}_i)) = \mathit{code}(\mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i) = \mathbf{num}(\mathbf{k}_j)).$$

Step 3. Imp-encoded transitivity of equality chains Steps 1 and 2 through the common middle term $\mathbf{num}(\mathbf{k}_j)$:

$$\vdash (\mathbf{thmT}(\mathbf{x}) = \mathbf{k}_j) \supset (\mathbf{thmT}(g(\mathbf{x})) = \mathit{code}(\mathbf{thmT}(\mathbf{num}(\mathbf{x})) = \mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i))).$$

Step 4. The term

$$K_{\mathbf{part}}(\mathbf{x}) := \langle \mathbf{k}_{\mathbf{tag_sb}}, \langle \mathbf{k}_1, \mathbf{num}(\mathbf{x}) \rangle, \mathbf{x} \rangle$$

is the encoded single-variable substitution that, under the diagonal identity $\mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i) = \mathit{code}(\mathbf{G})$, satisfies

$$\vdash (\mathbf{thmT}(\mathbf{x}) = \mathbf{k}_j) \supset (\mathbf{thmT}(K_{\mathbf{part}}(\mathbf{x})) = \mathit{code}(\neg(\mathbf{thmT}(\mathbf{num}(\mathbf{x})) = \mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i)))).$$

Step 5a. Let $t' := \mathit{code}(\mathbf{axExFalse} \mathbf{A} \perp)$ be the encoded derivation of Guard’s schema $\mathbf{A} \supset (\neg \mathbf{A} \supset \perp)$ with $\mathbf{A} \equiv (\mathbf{x}_0 = \mathbf{x}_1)$ and $\perp \equiv (\mathbf{O} = \mathbf{s}\mathbf{O})$. Set $h(\mathbf{x})$ to be *two nested single-variable substitution wraps* of t' (var 0 outermost, var 1 innermost):

$$h(\mathbf{x}) := \langle \mathbf{k}_{\mathbf{tag_sb}}, \langle \mathbf{k}_0, \mathit{code}(\mathbf{thmT}(\mathbf{num}(\mathbf{x}))) \rangle, \langle \mathbf{k}_{\mathbf{tag_sb}}, \langle \mathbf{k}_1, \mathit{code}(\mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i)) \rangle, t' \rangle \rangle.$$

$\mathbf{thmT}$ decodes the outer wrap by its single `sb`-clause into $\mathbf{sbF}(\langle \mathbf{k}_0, \mathit{code}(\mathbf{thmT}(\mathbf{num}(\mathbf{x}))) \rangle, \cdot)$ applied to the inner wrap, which decodes likewise into $\mathbf{sbF}(\langle \mathbf{k}_1, \mathit{code}(\mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i)) \rangle, \mathbf{thmT}(t'))$.

Both substituents are **num**-based (var-free) codes, hence inert under the outer re-scan (§6); writing $E := \mathbf{thmT}(\mathbf{num}(\mathbf{x})) = \mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i)$,

$$\vdash \mathbf{thmT}(h(\mathbf{x})) = \mathit{code}(E \supset \neg E \supset (\mathbf{O} = \mathbf{s} \mathbf{O})).$$

Step 5b. Two applications of the Carneiro-lifted encoded modus ponens **imp_encoded_mp** compose Steps 3, 4, and 5a to produce

$$\vdash (\mathbf{thmT}(\mathbf{x}) = \mathbf{k}_j) \supset (\mathbf{thmT}(\mathbf{bigterm}(\mathbf{x})) = \mathit{code}(\mathbf{O} = \mathbf{s} \mathbf{O})),$$

where

$$\mathbf{bigterm}(\mathbf{x}) := \langle \mathbf{k}_{\text{tag_mp}}, \langle \mathbf{k}_{\text{tag_mp}}, h(\mathbf{x}), g(\mathbf{x}) \rangle, K_{\text{part}}(\mathbf{x}) \rangle.$$

Final assembly. Instantiate the universal Step 5 at $\mathbf{x} = \mathbf{x}_1$. From $\vdash \mathit{Con}_T$ by Rule III applied to $\mathbf{bigterm}(\mathbf{x}_1)$ we obtain

$$\vdash \neg(\mathbf{thmT}(\mathbf{bigterm}(\mathbf{x}_1)) = \mathit{code}(\mathbf{O} = \mathbf{s} \mathbf{O})).$$

The classical contrapositive **axContrapos** then yields

$$\vdash \neg(\mathbf{thmT}(\mathbf{x}_1) = \mathbf{k}_j).$$

The predicate-Leibniz rule **substF_cong** applied at the formula

$$\Phi \equiv \neg(\mathbf{thmT}(\mathbf{x}_1) = \mathbf{x}_2)$$

with the equation $\vdash \mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i) = \mathbf{k}_j$ converts this to $\vdash \mathbf{G}$. An inlined re-derivation of Theorem 11 (Gödel I) for the sub-form of $\mathbf{G}$ then yields $\vdash (\mathbf{O} = \mathbf{s} \mathbf{O})$.

5 Implicit facts in Guard’s text, made explicit

The formalisation forced a number of small but non-trivial corrections, disambiguations, or additions to Guard 1963.

5.1 Underline notation: the internal num versus the meta-encoding $\mathit{code}(\cdot)$

Guard’s text uses an underline notation $\underline{\mathbf{x}}$ whose meaning shifts between (i) the internal T -term $\mathbf{num}(\mathbf{x})$ and (ii) the meta-level Gödel handle $\mathit{code}(\mathbf{x})$. These are genuinely different objects:

- $\mathbf{num}(\mathbf{x})$ is a T -term — the unary functor $\mathbf{num} \in \mathit{Fun}_1$ applied to $\mathbf{x}$. It *computes*, inside T , the encoding of the value of $\mathbf{x}$.
- $\mathit{code}(\mathbf{x})$ is the meta-level encoding of the *syntactic* object $\mathbf{x}$ (the Gödel handle), a closed term computed outside T .

They are connected only by the Deriv-level closure on closed numerals:

$$\vdash \mathbf{num}(\mathbf{k}_n) = \mathit{code}(\mathbf{k}_n) \quad (\text{the lemma } \mathit{numEq} \, n),$$

and they differ for non-numeral $\mathbf{x}$ (e.g. for a variable $\mathbf{x}_0$, the term $\mathbf{num}(\mathbf{x}_0)$ is open whereas $\mathit{code}(\mathbf{x}_0) = \langle \mathbf{k}_{\text{tag_var}}, \mathbf{k}_0 \rangle$ is a fixed closed handle). Note that *neither* of these equals $\mathbf{x}$ itself: in particular $\mathbf{num}(\mathbf{s} \mathbf{O}) = \mathit{code}(\mathbf{s} \mathbf{O}) \neq \mathbf{s} \mathbf{O}$, so the underline is never the identity.

In the Agda development the file **Thm14F.agda** introduces the *slot abbreviation* code $\mathbf{t} := \mathbf{num}(\mathbf{t})$ (its code is literally **num**), used in the encoded positions where Guard writes $\underline{\mathbf{x}}$. This abbreviation coincides with the meta-encoding $\mathit{code}(\cdot)$ only on closed numerals — precisely via numEq — and the formalisation makes every such bridge explicit (e.g. $\vdash \mathbf{num}(\mathbf{k}_i) = \mathit{code}(\mathbf{k}_i)$ at the diagonal numeral i).

5.2 The two G’s of the diagonal lemma

There are two distinct formulas, both called “G” in informal treatments of the diagonal lemma:

$$\begin{aligned}\mathbf{G}_{\text{sub}} &\equiv \neg(\mathbf{thmT}(\mathbf{x}_1) = \mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i)), \\ \mathbf{G}_{\text{sbf}} &\equiv \neg(\mathbf{thmT}(\mathbf{x}_1) = \mathbf{sbf}(\langle \mathbf{k}_0, \mathbf{num}(\mathbf{k}_i) \rangle, \mathbf{k}_i)).\end{aligned}$$

These are equal in T only up to the closure **sub_eq**, which is a term-level T -derivable equation, *not* a Formula-syntactic equality. Consequently, a proof of Gödel I phrased for $\mathbf{G}_{\text{sbf}}$ (as in our `T4.GoedelI`) does not literally apply to $\mathbf{G}_{\text{sub}}$. The formalisation bridges this gap either by inlining a copy of Gödel I for $\mathbf{G}_{\text{sub}}$ (the route taken in `Thm14GoedelIII`), or by invoking the *predicate-Leibniz* rule **substF_cong** to transfer derivability across the term-equation $\mathbf{sub}(\mathbf{k}_i, \mathbf{k}_i) = \mathbf{k}_j$. Guard’s text treats the two G’s as interchangeable without comment.

5.3 Modus-ponens slot order

Guard writes $mp(\text{antecedent}, \text{implication})$ in his encoded-derivation tables, while our $\langle \mathbf{k}_{\text{tag_mp}}, \text{impl}, \text{ant} \rangle$ uses the opposite order.

Guard’s expression for *bigterm*($\mathbf{x}$),

$$4J[4J(J(\mathbf{num}\ \mathbf{x}, 1), \mathbf{x}) + 1, 4J(g(\mathbf{x}), h(\mathbf{x})) + 2] + 2,$$

thus hides which sub-slot is which, and a careful sign-by-sign translation is needed.

5.4 Closedness side conditions for Rule III

Rule III (*substT*, *substF*) only reduces $\text{substT}(\mathbf{x}_a, \mathbf{t}, \mathbf{t}')$ when $\mathbf{x}_a$ and $\mathbf{t}'$ match by definitional pattern. For closed sub-terms (numerals, the encoded false formula $\text{code}(\mathbf{O} = \mathbf{s}\ \mathbf{O})$, the diagonal numeral $\mathbf{k}_i$) the formalisation requires explicit *Closed* witnesses and bridges via **closedAt**. Guard’s text uses “ $\mathbf{k}_i$ is a numeral so its substitution is trivial”. The Agda development must construct, e.g.

$$\text{closed_codeFalse} : \text{Closed } \text{code}(\mathbf{O} = \mathbf{s}\ \mathbf{O})$$

as a nested $\langle \cdot \rangle$ /numeral proof tree.

5.5 The validating-decoder invariant on thmT

Guard’s verifier **thmT** is described informally as “returns the code of the proved formula”. But for inputs that are *not* the encoding of a derivation, **thmT** must still be defined and must still satisfy the closure equations used in the proof (e.g. $\vdash \mathbf{thmT}(\text{code}(\text{mp}(p, q))) = \dots$). We adopt the convention

$$\vdash \mathbf{thmT}(\text{garbage}) = \text{code}(\mathbf{O} = \mathbf{O}),$$

(the canonical trivial truth), which makes the closure equations **thmT_at_mp**, **thmT_at_sb**, etc. universal in their raw $\langle \cdot \rangle$ -encoded inputs. This invariant is implicit in Guard but essential to make the closures usable.

5.6 The encoded t' schema

Guard writes “ t' is the encoded derivation of $\mathbf{x}_0 = \mathbf{x}_1 \supset \mathbf{x}_0 \neq \mathbf{x}_1 \supset \mathbf{O} = \mathbf{s}\ \mathbf{O}$ ”. The formalisation exhibits this concretely as

$$t' = \text{code}(\mathbf{axExFalso}(\mathbf{x}_0 = \mathbf{x}_1)(\mathbf{O} = \mathbf{s}\ \mathbf{O}))$$

with **axExFalso** built from **axK**, **axS**, **axNeg**, and a completeness handle **thmT_complete_rec** proving the verifier’s expected output. Guard glosses both pieces.

5.7 Carneiro lift for hypothetical derivations

Guard frequently asserts implications of the form “under the hypothesis $\mathbf{P}$ we have $\mathbf{A}$ and $\mathbf{B}$, hence $\mathbf{C}$ ”. In a pure Hilbert calculus there is no implication-introduction rule, and our `Deriv` is *context-free* (`Deriv : Formula → Set`, no hypothesis context), so the only way to express “under hypothesis $\mathbf{P}$ ” is the object formula $\mathbf{P} \supset (\dots)$, and the only way to *compose* such conditionals is to lift every ordinary combinator to its $\mathbf{P} \supset (\dots)$ form via Carneiro’s **axS/axK** technique [4]. The formalisation extracts this into a small toolkit in `T4.Thm12.ImpHelpers`:

`impLift`, `impMp`, `impCong1`, `impCongL`, `impCongR`, `impRuleSym`, `impEqTrans`,

together with the encoded modus ponens `imp_encoded_mp` (`T4.Thm12.EncodedMp`).

Is it essential? Relative to the context-free `Deriv` representation, yes: the theorems of the Theorem 14 argument are intrinsically conditional — Step 5 is literally $\text{th}(\mathbf{x}) = j \supset \text{th}(\text{bigterm}(\mathbf{x})) = \text{code}(\mathbf{O} = \mathbf{sO})$ — and this conditional is exactly what is combined with `ConT` to produce $\mathbf{G}$, so it cannot be restructured away. It is *not* essential in the absolute sense: a deduction theorem for BRA is admissible ([2, Exercise 19]), so one could instead carry a hypothesis context in `Deriv` and prove the deduction theorem once. The Carneiro lift is the *pointwise* deduction theorem — lift the combinators actually used, rather than prove the general metatheorem — and given our context-free judgement it is the lightweight, effectively forced choice.

A concrete example where it helped. The step in `thmT`’s Step 4 (`T4.Thm.Thm14Step4`) that *uses the Gödel hypothesis itself*. Under $\mathbf{P} = (\text{th}(\mathbf{x}) = j)$ one must rewrite `sbF(cSpec, th(x))` into `sbF(cSpec, j)` — i.e. apply the assumption “ $\text{th}(\mathbf{x}) = j$ ” *inside* a congruence, without discharging it:

```
hyp_imp : Deriv (imp P (eqF (ap1 thmT x) j))
hyp_imp = impRefl P -- assuming P, P holds: i.e. assuming th(x)=j, th(x)=j
```

```
step_B_imp : Deriv (imp P (eqF (ap2 sbf cSpec (ap1 thmT x)) (ap2 sbf cSpec j)))
step_B_imp = impCongR {P} sbf (ap1 thmT x) j cSpec hyp_imp
```

Here `impRefl P` extracts the hypothesis as $\mathbf{P} \supset \mathbf{P}$ — and $\mathbf{P}$ is the equation $\text{th}(\mathbf{x}) = j$ — while `impCongR` threads it through the congruence of `sbF`, yielding $\mathbf{P} \supset (\text{sbF}(cSpec, \text{th}(\mathbf{x})) = \text{sbF}(cSpec, j))$. Unfolded, this is just `ax_eqCongR` wrapped with `axS/axK`; the toolkit makes it one line.

The same proof shows the lift is doing genuine work of two distinct kinds at once. The surrounding steps A, C, D of Step 4 are *unconditional* facts (the closure `thmT_at_sb`, the diagonal bridge `codeFormulaG_eq_j`, the per-shape `sbF/sbT` evaluations); each is carried past $\mathbf{P}$ by `impLift` (a bare `axK` wrap). Only step B *consumes* the hypothesis (`impRefl P + impCongR`). And `impEqTrans` chains all four equalities under the single $\mathbf{P}$ to conclude $\mathbf{P} \supset (\text{th}(K_{\text{part}}(\mathbf{x})) = \text{code}(\neg(\text{th}(\mathbf{x}) = \text{sub}(\mathbf{k}_i, \mathbf{k}_i))))$. Carrying unconditional facts past $\mathbf{P}$ and applying $\mathbf{P}$ where needed is exactly the bookkeeping a deduction theorem would otherwise perform; the Carneiro lift does it pointwise. Its headline consumer is `imp_encoded_mp`: the two encoded modus-ponens steps of Step 5b are composed entirely under P_x via `imp_thmT_at_mp + impEqTrans`. Guard never names the principle.

5.8 Predicate-Leibniz substitutivity

Guard’s transition from “ $\neg(\text{thmT}(\mathbf{x}_1) = \mathbf{k}_j)$ ” to “ $\neg(\text{thmT}(\mathbf{x}_1) = \text{sub}(\mathbf{k}_i, \mathbf{k}_i))$ ” is stated as an obvious rewrite. In a Hilbert calculus this requires the *predicate-Leibniz* principle

$$\vdash \mathbf{a} = \mathbf{b} \implies \vdash \text{Sub}(\text{code}(\Phi), \text{code}(\mathbf{x}_k), \text{code}(\mathbf{a})) \supset \text{Sub}(\text{code}(\Phi), \text{code}(\mathbf{x}_k), \text{code}(\mathbf{b})),$$

which is itself a small but non-trivial Hilbert-meta-theorem (proved by induction on Φ , using **axContrapos** on the negative slot; see **BRA3.Substitutivity**). Guard does not mention it.

5.9 Three small typographical issues

- Guard writes “ j ” in places where his own underline convention requires $\underline{j}$ ($= \mathbf{num} \mathbf{k}_j = \mathit{code}(\mathbf{k}_j)$). The formalisation makes the distinction explicit at every occurrence.
- Definition 12 conflates the underline notation with the numeral. The formalisation requires the precise statement that $\vdash \mathbf{num}(\mathbf{k}_n) = \mathit{code}(\mathbf{k}_n)$ for every meta-natural n — i.e. **num** maps a numeral to its *encoding*, not to itself — instantiated through the lemma *numEq*. In particular $\mathbf{num}(\mathbf{s} \mathbf{O}) = \mathit{code}(\mathbf{s} \mathbf{O}) \neq \mathbf{s} \mathbf{O}$. The text leaves this implicit and the underline notation makes it easy to mistake $\mathbf{num}(\mathbf{x})$ for $\mathbf{x}$ itself.
- In the description of $h(\mathbf{x})$, Guard does not commit to a representation of the substitution. The formalisation realises the two-variable effect as two *nested single-variable* wraps $\langle \mathbf{k}_{\text{tag_sb}}, \langle \mathbf{k}_0, S_0 \rangle, \langle \mathbf{k}_{\text{tag_sb}}, \langle \mathbf{k}_1, S_1 \rangle, t \rangle \rangle$ and uses the numeral-inertness lemma (§6) to pass the inner substituent S_1 through the outer re-scan.

6 Single-variable substitution suffices: numeral-inertness

The verifier’s substitution clause is the *single-variable* encoded substitution **sb**f/**sb**t (cf. Guard’s **sb**, Def. 16). Both the compound-functor congruences feeding Theorem 12 and the two-variable wrap of Step 5a require, on the face of it, a *simultaneous* substitution of several variables at once. We obtain all of them from *nested single-sb wraps*, with no dedicated multi-variable functor. The one fact that makes this work is a numeral-inertness lemma; we record it here because it is exactly the place Guard’s text leaves the argument implicit, and because the alternative (a genuine simultaneous-substitution primitive) carries a different proof-theoretic commitment relevant to one of Guard’s open problems.

The apparent obstruction. A nested wrap substitutes the variables one at a time, so an outer pass re-scans a substituent already plopped by an inner pass. In Theorem 12’s compound cases the substituents are $\mathbf{num} X$ for the *object* variable X , and for open X (e.g. $X = \mathbf{x}_1$) the term $\mathbf{num}(\mathbf{x}_1)$ does not reduce: $\mathbf{num} = \mathbf{C} \mathit{numAux} \mathbf{o} \mathbf{u}$ with $\mathit{numAux} = \mathbf{R} \mathbf{o} \mathit{numStep} \mathbf{v}$, whose **R**-recursion fires only when its second argument is syntactically **O** or $\mathbf{s}(\cdot)$; a variable is neither, so $\mathbf{num}(\mathbf{x}_1)$ stays a stuck open application. Since **sbt** dispatches on $\mathit{get_tag}(\mathit{input})$ and that tag is an *open* expression on the stuck term, naive evaluation of $\mathbf{sbt}(\langle \mathbf{k}_k, S \rangle, \mathbf{num}(\mathbf{x}_1))$ never settles. Read *computationally*, the re-scan stalls — which is why a simultaneous plop (no re-scan) looks unavoidable.

The numeral-inertness lemma. The obstruction conflates computation with *derivability*. We never need **sbt** to *compute* on a stuck $\mathbf{num}(\mathbf{x}_1)$; we need the *equation*

$$\vdash \mathbf{sbt}(\langle \mathbf{k}_k, S \rangle, \mathbf{num} X) = \mathbf{num} X \quad \text{for an arbitrary object term } X$$

(the Agda lemma **sbt_num_inert**, T4.NumInert), and this is a *theorem* of the object system, proved by *internal* induction on X (Rule III / **ruleIndNat**), not by inspection of syntax. The base case is $\mathbf{sbt}(\dots, \mathbf{num} \mathbf{O}) = \mathbf{sbt}(\dots, \mathbf{O}) = \mathbf{O} = \mathbf{num} \mathbf{O}$; the step uses $\mathbf{num}(\mathbf{s} X) = \langle \mathbf{k}_{\text{tag_ap}_1}, \langle \mathbf{k}_{\text{tag_s}}, \mathbf{num} X \rangle \rangle$ (the lemma **num_at_S**), which at *each* inductive step rewrites the numeral to a $\langle \cdot \rangle$ -code the descent can enter, discharging the recursive call by the induction hypothesis. So the descent never meets a bare $\mathbf{num}(\mathbf{x}_1)$: it meets a planted num-leaf and passes through it. Lifting through the **ap**₁/**ap**₂ code-wrappers (the predicate *NumCode* in

T4.SbStep), *every* substituent built from numeral leaves is inert under *every* single-variable substitution.

Intuition: the internalisation of “ S^n0 is closed”. The lemma says nothing more than the elementary fact that a numeral is a *closed* term: S^n0 contains no variable, so every substitution acts as the identity on it. For a concrete numeral this is immediate — its code is a var-free $code(\cdot)$ -tree with no $\langle k_{tag_var}, \cdot \rangle$ leaf, so the encoded re-scan visibly passes through. The only subtlety, and the reason an internal induction is needed at all, is the *generic* statement: for a variable X the term **num** X is a *stuck open* application, not literally a numeral, so its closedness is invisible to the tag-dispatch and must be *promoted to an internal theorem*. **ruleIndNat** is precisely that promotion: it internalises “**num** X is (the code of) a closed numeral, for all X ” as a derivable equation.

Consequence: the multi-variable machinery is eliminated. With inertness in hand, each compound-functor congruence of Theorem 12 — e.g. specialising the binary congruence at the *open* terms $g(\underline{x}), g(\underline{x}), h(\underline{x}), h(\underline{x})$ in building $D_{C(f,g,h)}$ — and the Step 5a wrap of Theorem 14 are realised by *nested single-sb wraps*: an inner pass plants a substituent, an outer pass plants the next and re-scans the inner one, which inertness leaves fixed. Concretely **thmT** decodes a k -fold nested $\langle k_{tag_sb}, \dots \rangle$ wrap by k applications of its single **sb**-clause into **sb**f(spec₀, **sb**f(spec₁, $\dots$)). The dedicated two- and three-variable functors **sbt**₂, **sb**f₂, **sbt**₃, **sb**f₃, their contract records, and the two extra discriminator levels in the **thmT** cascade — roughly ten thousand lines across a dozen files — are thereby *removed*. This recovers exactly Guard’s own explicit Theorem 14 construction (guard15.pdf p. 17), where the multi-variable effect is obtained by *nested single-sb steps with the open substituent applied last*; what Guard left implicit — that the re-scanned substituents genuinely pass through — is the inertness lemma.

The induction-avoiding alternative, and an open problem. There is a second, equally sound route that we deliberately keep on record. A genuine *simultaneous* substitution functor **sub**₂/**sub**₃ plops all substituents at once; nothing is re-scanned, so its closure proofs need *no* internal induction. (It is sound: its encoded closures are the encoded-level counterpart of the *derived admissible* rule **ruleInst2**, $k_1 \neq k_2 \Rightarrow \vdash \mathbf{P} \Rightarrow \vdash simSubstF(k_1, t_1, k_2, t_2, \mathbf{P})$ — Church’s Standard Metatheorem VII, *fully derived* from single substitution by α -renaming through a fresh variable in **BRA3.RuleInst2** — so it adds no proof-theoretic strength.) The contrast is precise: the *abstract* rule reduces to single substitution on raw formulas, but the *encoded* simultaneous functor does not reduce to iterated encoded **sbt** *by computation* (the re-scan stalls) — only *by derivability*, and that derivation is exactly **sbt.num.inert**, which uses internal induction. So the two routes differ in *where they spend induction*: the inertness route puts an internal induction into the verifier’s substitution-correctness; the simultaneous-substitution route keeps that piece induction-free.

This bookkeeping is not idle. Guard (guard19.pdf, p. 19, Def. 14 and the OPEN PROBLEM following Theorem 18) defines the *induction-free* fragment BRA_0 — the variable-free instances of **BRA**’s axioms with *modus ponens as the only rule* — whose verifier **th**₀ has *only* an axiom clause and an mp clause (no substitution clause, no induction clause), and *conjectures* that its consistency **th**₀(y) $\neq code(\mathbf{O} = \mathbf{s} \mathbf{O})$, though valid, is *unprovable in BRA* — a putative counterexample to the usual provable consistency of induction-free fragments. Whether, and where, a verifier’s substitution machinery can be made induction-free is exactly the kind of distinction that question turns on; the inert-lemma versus simultaneous-substitution choice above is one concrete instance, which is why we keep the simultaneous functors on record even though the shipped development uses the single-variable route.

7 Conclusion

The formalisation closes Guard’s argument for Gödel’s second incompleteness theorem for BRA in a completely concrete, constructively-verified form, and clarifies the implicit mathematical machinery (closedness side conditions, predicate-Leibniz, Carneiro lift, validating-decoder invariant, the two-forms-of-**G** distinction) on which Guard’s text relies without comment. As §6 documents, the multi-variable effects needed in Theorems 12 and 14 are obtained from *nested single-variable substitution*, the re-scan of each numeral-based substituent being discharged by the numeral-inertness lemma **sbt_num_inert** — the internalisation, by Rule III, of the fact that a numeral is closed. This recovers Guard’s own explicit construction (guard15.pdf p. 17) and eliminates the dedicated two- and three-variable simultaneous-substitution machinery. We keep on record, as a remark bearing on Guard’s open problem about the consistency of the induction-free fragment BRA_0 (guard19.pdf p. 19), that the simultaneous-substitution route is the *induction-free* alternative to the inertness lemma: the two agree on the encoded result but differ on whether the verifier’s substitution-correctness is proved with or without internal induction.

Acknowledgements. The Carneiro-lift toolkit follows Mario Carneiro’s natural-deduction note [4]. The course-of-values infrastructure (the Agda modules `BRA3.CourseOfValues`, `BRA3.PairAlgebra`, and `BRA3.RuleInst2`) builds on Church’s system as transmitted by Guard [2, 1].

References

- [1] A. Church. *Basic recursive arithmetic* (the system of functors **s**, **o**, **u**, **v**, **C**, **R** and the axioms of §1.3). Lectures, Logic Seminar, Princeton University, 1957–1959. The Gödel numbering and the first incompleteness theorem for the system are, in Guard’s words, “essentially those of Church” [2, p. 13].
- [2] R. B. Guard. *Lecture Notes on Recursive Arithmetic*. 1963. (The working excerpt used here, `guard15.pdf`, contains Definitions 6–16, Exercises 17–24, and Theorems 8–16, including the verifier **th** (Def. 16) and Gödel’s first and second theorems for BRA (Thms. 11 and 14).)
- [3] J. R. Shoenfield. *Mathematical Logic*. Addison–Wesley, 1967. (Chapter 6, §§6.6–6.7: expression numbers and representability; the source of the `code(u)/⟨·⟩/Sub/Num/th` notation followed throughout.)
- [4] M. Carneiro. *Natural Deduction in the Metamath Proof Language*. <https://us.metamath.org/occat/natded.pdf>. The `axS/axK` technique behind `T4.Thm12.ImpHelpers` and `imp_encoded.mp`.